



NeuroSky

MindWave Mobile · Apple SDK

Developer Guide

Version 1.0.0

May 2026

Real-time EEG integration for iOS and macOS
via Bluetooth Low Energy · BT Classic on macOS
Built on Swift Concurrency & AsyncStream

Table of Contents

1.	Overview	3
2.	How It Works — Architecture	4
3.	Requirements	6
4.	Installation	7
5.	Permissions	8
6.	Quick Start	9
7.	EEG Data Model	11
8.	EEG Frequency Bands Explained	12
9.	Signal Quality	13
10.	Commands	14
11.	Simulator — Develop Without Hardware	15
12.	Error Handling & Reconnection	16
13.	Advanced Patterns	18
14.	Troubleshooting	22
15.	Testing	23
16.	API Reference	24

1. Overview

The NeuroSky MindWave Mobile Apple SDK is a modern Swift library that lets you read real-time EEG (electroencephalography) data from a NeuroSky MindWave Mobile headset on iOS 14+ and macOS 11+.

The SDK is built around Swift Concurrency — connection, command, and data-stream APIs are all async, and EEG packets are delivered through `AsyncStream<BrainWaveData>` so they integrate naturally with SwiftUI's `.task { }` modifier and `for await` loops.

Key features

Feature	Description
BLE + BT Classic	iOS: BLE only. macOS: BLE (default) or BT Classic via mode: <code>.btClassic</code>
Swift Concurrency	<code>AsyncStream<BrainWaveData></code> — natural fit with SwiftUI <code>.task { }</code>
Built-in Simulator	Full data simulation without any hardware — develop without a headset
SPM distribution	One-line <code>Package.swift</code> dependency, no extra build setup
iOS 14+ / macOS 11+	Wide platform coverage; <code>@MainActor</code> isolation for safe UI updates

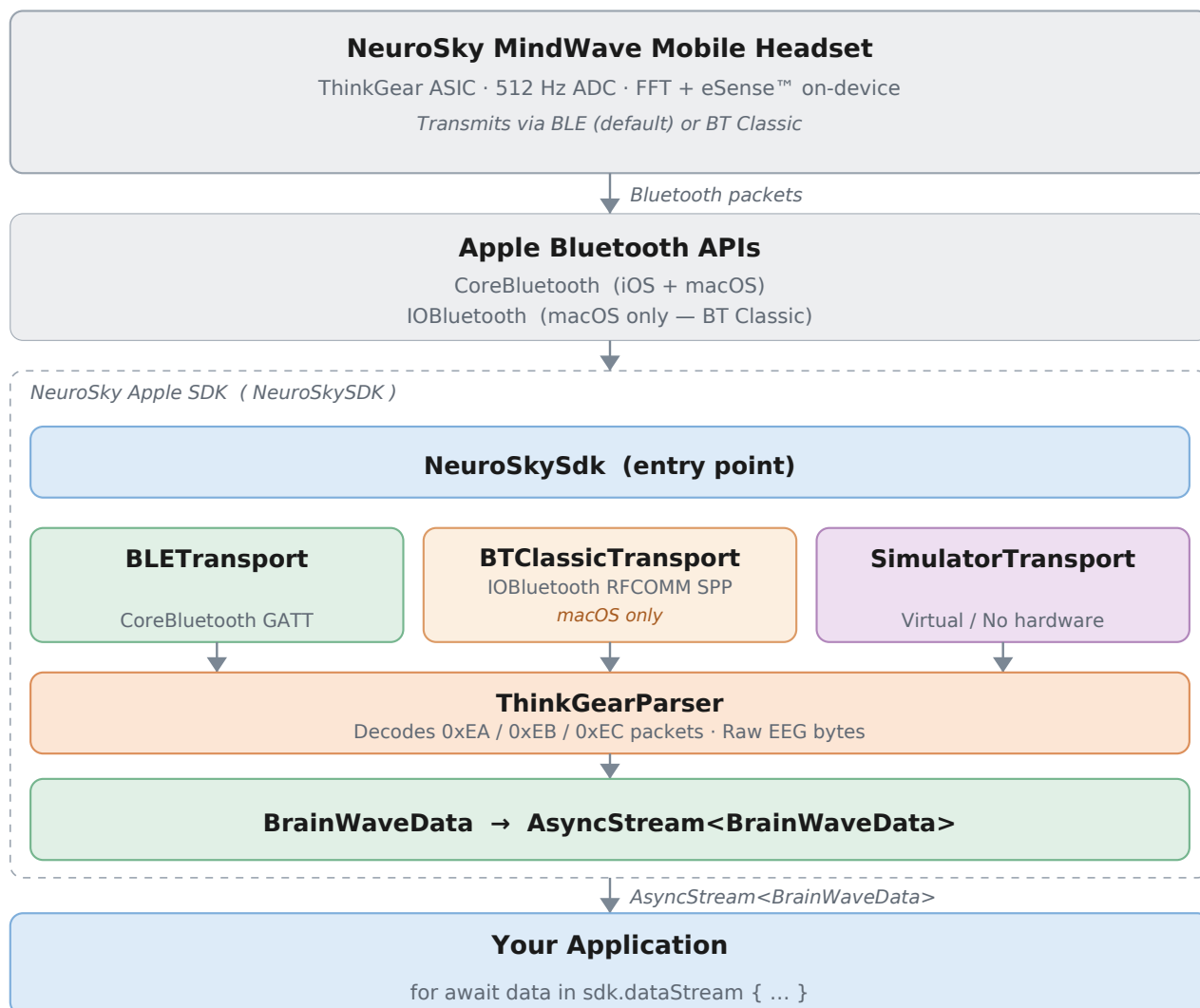
What you can measure

The MindWave Mobile headset contains a single dry electrode on the forehead (FP1 position) and a reference clip on the ear. From this single channel, the ThinkGear ASIC chip on board computes:

- Raw EEG waveform — 512 samples/sec, signed 16-bit values
- 8 frequency band powers — Delta, Theta, Alpha (Low/High), Beta (Low/High), Gamma (Low/Mid)
- eSense™ Attention — NeuroSky's proprietary attention index (0~100)
- eSense™ Meditation — NeuroSky's proprietary relaxation index (0~100)
- Signal quality — 0 (perfect contact) to 200 (no signal)

2. How It Works — Architecture

Understanding the data flow — from the BLE radio in the headset all the way to your for await loop — helps you design your app and debug issues effectively.



BLE vs BT Classic — internal differences

BLE (iOS + macOS): The MindWave Mobile exposes three GATT characteristics. The SDK subscribes to notifications on the eSense (039afff8) and RawEEG (039afff4) characteristics, then writes the handshake byte to the handshake characteristic (039affa0) to start data flow. No pairing required.

BT Classic (macOS only): The MindWave Mobile emulates a serial port over SPP (UUID 00001101-...). The SDK opens an IOBluetoothRFCOMMChannel and reads a continuous byte stream. ThinkGearParser synchronizes on the 0xAA 0xAA sync header. The device must be paired in macOS Bluetooth settings first.

Both paths produce identical BrainWaveData output through the same dataStream.

Transport selection

NeuroSkySdk.connect(_:mode:) accepts an explicit TransportMode:

```
// BLE – default, works on iOS + macOS, no pairing required
```

```
try await sdk.connect("MindWave Mobile")           // mode: .ble (implicit)
try await sdk.connect("MindWave Mobile", mode: .ble)

// BT Classic – macOS only, headset must be paired in System Settings first
try await sdk.connect("MindWave Mobile", mode: .btClassic)
```

On iOS, passing mode: `.btClassic` throws `TransportError.btClassicNotAvailableOniOS`.

3. Requirements

Platform requirements

Component	Minimum
iOS	14.0
macOS	11.0 (Big Sur)
Swift	5.7+
Xcode	14+
Bluetooth	BLE adapter (BLE mode) or Classic BT (BT Classic mode, macOS only)
Pairing	Not required for BLE; required for BT Classic

Supported headset

This SDK is designed and tested for the NeuroSky MindWave Mobile 2. Both BLE and BT Classic modes are supported on macOS. iOS supports BLE only — this is a platform limitation, not an SDK limitation: Apple does not expose RFCOMM SPP on iOS for non-MFi accessories.

Note: The original (non-Mobile) MindWave headset is not supported. Use the MindWave Mobile 2 (commonly labeled simply "MindWave Mobile").

4. Installation

Option A — Swift Package Manager via Xcode

- Open your project in Xcode
- File → Add Package Dependencies...
- Enter the URL: <https://github.com/nsk-bci/mindwave-sdk-apple>
- Select version rule: Up to Next Major from 1.0.0
- Click Add Package
- In the product picker, add the NeuroSkySDK library to your target

Option B — Swift Package Manager via Package.swift

For library targets or non-Xcode SwiftPM projects, add to Package.swift:

```
// swift-tools-version:5.7
import PackageDescription

let package = Package(
    name: "MyApp",
    platforms: [
        .iOS(.v14),
        .macOS(.v11)
    ],
    dependencies: [
        .package(
            url: "https://github.com/nsk-bci/mindwave-sdk-apple",
            from: "1.0.0"
        )
    ],
    targets: [
        .target(
            name: "MyApp",
            dependencies: [
                .product(name: "NeuroSkySDK", package: "mindwave-sdk-apple")
            ]
        )
    ]
)
```

Then resolve and build:

```
swift package resolve
swift build
```

Verifying installation

Add a single import to confirm the package resolves:

```
import NeuroSkySDK
```

If Xcode reports No such module 'NeuroSkySDK', ensure the package appears under the Frameworks, Libraries, and Embedded Content section of your target settings — adding the package alone is not enough; you also need to link the library to your target.

5. Permissions

Both iOS and macOS require an explicit Bluetooth usage description so the OS can show a permission prompt to the user.

iOS — Info.plist

Add the Bluetooth usage description to your app's Info.plist:

```
<key>NSBluetoothAlwaysUsageDescription</key>
<string>Bluetooth is required to connect to the MindWave headset.</string>
```

iOS 13 and earlier also require `NSBluetoothPeripheralUsageDescription`, but this SDK targets iOS 14+ so only `NSBluetoothAlwaysUsageDescription` is needed.

CoreBluetooth will display a system permission dialog the first time your app initializes `CBCentralManager`. No additional runtime permission code is needed.

macOS — Info.plist + Entitlements

Info.plist:

```
<key>NSBluetoothAlwaysUsageDescription</key>
<string>Bluetooth is required to connect to the MindWave headset.</string>
```

App.entitlements (required for BT Classic via IOBluetooth):

```
<key>com.apple.security.device.bluetooth</key>
<true/>
```

Mac App Store distribution: Sandboxed macOS apps must include `com.apple.security.device.bluetooth` in their entitlements. Without it, IOBluetooth calls will silently fail — there is no permission dialog and no exception.

6. Quick Start

SwiftUI example

The most natural integration is inside a SwiftUI View with the `.task {}` modifier — the stream is automatically cancelled when the view leaves the hierarchy.

```
import SwiftUI
import NeuroSkySDK

struct ContentView: View {
    let sdk = NeuroSkySdk()
    @State private var attention: Int = 0
    @State private var meditation: Int = 0
    @State private var signal: String = "-"

    var body: some View {
        VStack(spacing: 20) {
            Text("Attention: \(attention)")
                .font(.largeTitle)
            Text("Meditation: \(meditation)")
                .font(.largeTitle)
            Text("Signal: \(signal)")
                .foregroundColor(signal == "good" ? .green : .red)
        }
        .task {
            do {
                try await sdk.connect("MindWave Mobile")
                try await sdk.setNotch60Hz() // Korea / USA
                for await data in sdk.dataStream {
                    attention = data.attention
                    meditation = data.meditation
                    signal = "\(data.signalQuality)"
                }
            } catch {
                print("Connection error: \(error)")
            }
        }
    }
}
```

UIKit example

For UIKit apps, manage the stream yourself via a Task you can cancel in `viewDidDisappear`:

```
import UIKit
import NeuroSkySDK

class ViewController: UIViewController {
    let sdk = NeuroSkySdk()
    var streamTask: Task<Void, Never>?

    override func viewDidLoad() {
        super.viewDidLoad()
        startStreaming()
    }

    func startStreaming() {
        streamTask = Task { @MainActor in
            do {
                try await sdk.connect("MindWave Mobile")
                try await sdk.setNotch60Hz()
                for await data in sdk.dataStream {
                    guard data.signalQuality != .noSignal else {
                        statusLabel.text = "No signal – adjust headset"
                        continue
                    }
                    attentionLabel.text = "Attention: \(data.attention)"
                    meditationLabel.text = "Meditation: \(data.meditation)"
                }
            } catch {
                statusLabel.text = "Error: \(error.localizedDescription)"
            }
        }
    }

    override func viewDidDisappear(_ animated: Bool) {
        super.viewDidDisappear(animated)
        streamTask?.cancel()
        Task { await sdk.disconnect() }
    }
}
```

Connecting by device name vs identifier

`connect()` accepts either a device name substring or a BLE peripheral UUID:

```
// By device name – BLE scan required (slower, ~5 sec)
try await sdk.connect("MindWave Mobile")

// By BLE peripheral UUID – direct connect, skips scan (faster, recommended)
try await sdk.connect("12345678-1234-1234-1234-123456789ABC")

// macOS BT Classic – by device name or Bluetooth address
try await sdk.connect("MindWave Mobile", mode: .btClassic)
try await sdk.connect("AA:BB:CC:DD:EE:FF", mode: .btClassic)
```

Use device name during prototyping when you don't yet know the identifier. For production, prefer caching the OS-assigned UUID — see Section 13 (`findDeviceIdentifier`) for the recommended pattern.

7. EEG Data Model

BrainWaveData is a value type (struct) emitted for every packet received from the headset.

```
public struct BrainWaveData {
    public let poorSignal: Int           // 0 = perfect, 200 = no signal
    public let attention: Int            // 0~100
    public let meditation: Int           // 0~100
    public let delta: Int                // 0.5~2.75 Hz
    public let theta: Int                // 3.5~6.75 Hz
    public let lowAlpha: Int             // 7.5~9.25 Hz
    public let highAlpha: Int            // 10~11.75 Hz
    public let lowBeta: Int              // 13~16.75 Hz
    public let highBeta: Int             // 18~29.75 Hz
    public let lowGamma: Int             // 31~39.75 Hz
    public let midGamma: Int             // 41~49.75 Hz
    public let rawEeg: [Int]             // 10 samples / packet at 512 Hz
    public var signalQuality: SignalQuality // computed property
}

public enum SignalQuality {
    case good           // poorSignal == 0
    case fair           // poorSignal 1~50
    case poor           // poorSignal 51~199
    case noSignal       // poorSignal == 200
}
```

Packet rate

Different ThinkGear packet types arrive at very different rates. Knowing which data type comes in which packet is essential for correctly using the SDK — see Section 13 for common filtering pitfalls.

Data type	Rate
eSense (attention, meditation)	~1 Hz
EEG frequency bands (delta ... midGamma)	~1 Hz
Raw EEG	~51 packets/sec (10 samples × 51 ≈ 512 Hz)

8. EEG Frequency Bands Explained

The MindWave Mobile's ThinkGear chip performs an FFT on the raw EEG waveform and outputs the power in 8 standard EEG frequency bands.

Band reference table

Band	Range	Associated with
Delta	0.5~2.75 Hz	Deep sleep, unconscious processes
Theta	3.5~6.75 Hz	Drowsiness, light sleep, creativity
Low Alpha	7.5~9.25 Hz	Relaxed, eyes-closed rest
High Alpha	10~11.75 Hz	Alert relaxation
Low Beta	13~16.75 Hz	Normal alert thinking
High Beta	18~29.75 Hz	Active thinking, concentration
Low Gamma	31~39.75 Hz	Cognitive binding, perception
Mid Gamma	41~49.75 Hz	Higher cognitive functions

Important — relative units: The values are in arbitrary relative power units, not calibrated to physical units ($\mu\text{V}^2/\text{Hz}$). Compare bands to each other (e.g., the ratio of alpha to beta) rather than treating any single value as an absolute measurement. Values are also not directly comparable across individuals or sessions.

9. Signal Quality

poorSignal ranges from 0 (perfect contact) to 200 (no signal). The SDK maps this to a SignalQuality enum so you can branch on a semantic value instead of comparing integers directly.

PoorSignal → SignalQuality mapping

poorSignal	SignalQuality	Meaning
0	.good	Excellent electrode contact
1~50	.fair	Acceptable signal
51~199	.poor	Poor contact — reposition headset
200	.noSignal	No electrode contact detected

Recommended handling

```
for await data in sdk.dataStream {
  switch data.signalQuality {
    case .noSignal:
      showAlert("No signal – place the headset on your forehead and clip to your ear")
    case .poor:
      showWarning("Poor signal – adjust the headset position")
    case .fair, .good:
      updateUI(data)
  }
}
```

Important: When poorSignal == 200, the attention and meditation values are meaningless (they will be 0). Always check signal quality before using eSense values.

Tips for improving signal quality

- Moisten the sensor tip — a small amount of water on the forehead sensor greatly improves conductance
- Clean the forehead — remove sunscreen, makeup, and sweat residue
- Adjust headset position — center the sensor on FP1 (just above the left eyebrow)
- Check the ear clip — the reference clip must make solid contact with the earlobe
- Wait 20~30 seconds after putting on the headset for eSense values to stabilize

10. Commands

Commands are sent to the headset via the handshake BLE characteristic after connection. The SDK exposes convenience methods for common commands plus a low-level entry point for raw bytes.

Convenience methods

```
// Notch filter – call immediately after connecting
// Removes 50Hz or 60Hz power-line interference from raw EEG
try await sdk.setNotch60Hz()      // Korea, USA, Japan (60Hz grid)
try await sdk.setNotch50Hz()      // China, Europe, Australia (50Hz grid)

// Raw EEG stream – disabled by default to save bandwidth
try await sdk.startRawEeg()
// ... collect data.rawEeg ...
try await sdk.stopRawEeg()
```

Low-level command access

```
try await sdk.sendCommand(NeuroSkyCommand.startEsense)
try await sdk.sendCommand(NeuroSkyCommand.stopEsense)
```

Command bytes reference

Constant	Value	Description
NeuroSkyCommand.startRawEeg	0x15	Enable raw EEG stream
NeuroSkyCommand.stopRawEeg	0x16	Disable raw EEG stream
NeuroSkyCommand.startEsense	0x17	Enable eSense stream (sent automatically on connect)
NeuroSkyCommand.stopEsense	0x18	Disable eSense stream
NeuroSkyCommand.notch50Hz	0x1B	50 Hz notch filter
NeuroSkyCommand.notch60Hz	0x1C	60 Hz notch filter

Tip: Without a notch filter, you will likely see a large 50Hz or 60Hz artifact in raw EEG and elevated Beta band values. Set the notch filter immediately after connecting, before reading data.

11. Simulator — Develop Without Hardware

SimulatorTransport generates realistic synthetic BrainWaveData at 1-second intervals without any Bluetooth hardware. It's especially valuable on the iOS Simulator (where CoreBluetooth is not available) and in CI pipelines.

Basic usage

```
// Initialize SDK in simulator mode
let sdk = NeuroSkySdk(simulator: .focused)

Task {
    try await sdk.connect("sim")    // address string is ignored
    for await data in sdk.dataStream {
        print("Attention: \(data.attention)")    // 70–95 in .focused mode
    }
}
```

Available modes

Mode	attention	meditation	poorSignal	Use case
.random	20~80	20~80	0~10	General data flow testing
.focused	70~95	40~60	0	Focused state UI testing
.relaxed	20~45	70~95	0	Relaxed state UI testing
.poorSignal	0	0	200	Signal loss and error handling

Restricting to debug builds

The simulator is available in both debug and release builds. Use `#if DEBUG` to restrict it to development if desired:

```
#if DEBUG
let sdk = NeuroSkySdk(simulator: .focused)
#else
let sdk = NeuroSkySdk()
#endif
```

iOS Simulator integration

CoreBluetooth does not function in the iOS Simulator (Xcode). The recommended pattern is to use `NeuroSkySdk(simulator:)` on the simulator and the real SDK on device:

```
#if targetEnvironment(simulator)
let sdk = NeuroSkySdk(simulator: .focused)
#else
let sdk = NeuroSkySdk()
#endif
```

12. Error Handling & Reconnection

Connection errors

`connect()` throws on failure. Wrap it in a do-catch:

```
Task {
    do {
        try await sdk.connect("MindWave Mobile")
    } catch BLEError.bluetoothUnavailable {
        showAlert("Bluetooth is off – enable it in Settings")
    } catch BLEError.connectionFailed {
        showAlert("Connection failed – is the headset on and nearby?")
    } catch BLError.deviceNotFound {
        showAlert("Device not found – pair it in macOS Bluetooth settings first")
    } catch TransportError.btClassicNotAvailableOniOS {
        showAlert("BT Classic is only supported on macOS – use BLE instead")
    } catch {
        showAlert("Unexpected error: \(error)")
    }
}
```

Automatic reconnection

Note: The SDK does not reconnect automatically after a disconnect. You must implement reconnection logic in your app.

```
func connectWithRetry(maxAttempts: Int = 3) {
    Task {
        for attempt in 1...maxAttempts {
            do {
                try await sdk.connect("MindWave Mobile")
                return // success
            } catch {
                if attempt < maxAttempts {
                    try? await Task.sleep(nanoseconds: 2_000_000_000) // wait 2 s
                } else {
                    showAlert("Failed after \(maxAttempts) attempts")
                }
            }
        }
    }
}
```

Observing connection state

Subscribe to `stateStream` alongside `dataStream` to react to connection lifecycle events:

```
Task {
    for await state in sdk.stateStream {
        switch state {
        case .scanning:    statusLabel = "Scanning..."
        case .connecting:  statusLabel = "Connecting..."
        case .connected:   statusLabel = "Connected"
        case .disconnected:
            statusLabel = "Disconnected"
            connectWithRetry()
        case .error(let err):
            statusLabel = "Error: \(err)"
        }
    }
}
```



```
}  
  }  
}
```

13. Advanced Patterns

Finding a device by name — findDeviceIdentifier

CoreBluetooth identifies peripherals by an OS-assigned UUID, not by Bluetooth MAC address. The UUID is stable for a given (app + device) pair, so the recommended pattern is: scan once, cache the UUID, then connect directly on subsequent launches to skip the scan delay.

```
// First launch – scan and save
if let uuid = await sdk.findDeviceIdentifier("MindWave Mobile", timeout: 10) {
    UserDefaults.standard.set(uuid, forKey: "mindwave_uuid")
    try await sdk.connect(uuid)
}

// Subsequent launches – connect directly (no scan needed)
if let uuid = UserDefaults.standard.string(forKey: "mindwave_uuid") {
    try await sdk.connect(uuid)
} else {
    // UUID not cached yet – fall back to name-based scan
    try await sdk.connect("MindWave Mobile")
}
```

Working with dataStream — packet timing and common pitfalls

Both the eSense and RawEEG characteristics share the same dataStream. They arrive at very different rates:

Source	Characteristic	Typical rate
eSense (attention, meditation, EEG bands)	039afff8	~1 Hz
Raw EEG samples	039afff4	~51 Hz (10 samples/packet × 512 Hz)

Do not filter by `attention == 0` — this silently drops all RawEEG packets because attention is always 0 in those packets:

```
// WRONG – drops ~98% of packets when RawEEG is enabled
for await data in sdk.dataStream {
    guard data.attention > 0 else { continue }
    processRawEeg(data.rawEeg) // ← never reached
}
```

Correct pattern — branch on the field you actually need:

```
// CORRECT – branch on the field you actually need
for await data in sdk.dataStream {
    if !data.rawEeg.isEmpty {
        processRawEeg(data.rawEeg) // RawEEG packet
    }
    if data.attention > 0 || data.meditation > 0 {
        updateUI(data) // eSense packet
    }
}
```

eSense only (without Raw EEG):

```
try await sdk.connect("MindWave Mobile") // startEsense sent automatically
for await data in sdk.dataStream where data.rawEeg.isEmpty {
    print("Attention: \(data.attention), Meditation: \(data.meditation)")
}
```

```
}
```

Raw EEG only:

```
try await sdk.connect("MindWave Mobile")
try await sdk.startRawEeg()
for await data in sdk.dataStream where !data.rawEeg.isEmpty {
    let samples = data.rawEeg    // 10 signed Int samples per packet at 512 Hz
    processDSP(samples)
}
```

Combine integration

For Combine-based UI architectures, expose the stream through `@Published` properties on an `ObservableObject`:

```
import Combine
import NeuroSkySDK

class EEGViewModel: ObservableObject {
    let sdk = NeuroSkySdk()
    @Published var attention: Int = 0
    @Published var meditation: Int = 0
    private var streamTask: Task<Void, Never>?

    func start() {
        streamTask = Task { @MainActor in
            try? await sdk.connect("MindWave Mobile")
            for await data in sdk.dataStream {
                self.attention = data.attention
                self.meditation = data.meditation
            }
        }
    }

    func stop() {
        streamTask?.cancel()
        Task { await sdk.disconnect() }
    }
}
```

Raw EEG recording

Accumulate raw samples in a buffer for offline analysis or playback. The example below collects 10 seconds ($512 \times 10 = 5120$ samples) at a time:

```
var samples: [Int] = []
try await sdk.startRawEeg()

for await data in sdk.dataStream {
    samples.append(contentsOf: data.rawEeg)
    if samples.count >= 512 * 10 { // 10 seconds of data at 512 Hz
        analyzeEEG(samples)
        samples.removeAll()
    }
}
```

Frequency band ratios

Because absolute band values are not calibrated, ratios between bands are often more meaningful. A common pattern is the alpha-to-beta ratio (often used as a rough indicator of relaxation versus focus):

```
for await data in sdk.dataStream {
    guard data.signalQuality == .good else { continue }

    let totalPower = data.delta + data.theta + data.lowAlpha +
                    data.highAlpha + data.lowBeta + data.highBeta
    guard totalPower > 0 else { continue }

    let alphaRatio = Double(data.lowAlpha + data.highAlpha) / Double(totalPower)
    let betaRatio = Double(data.lowBeta + data.highBeta) / Double(totalPower)
    print("Alpha/Beta ratio: \(alphaRatio / betaRatio)")
}
```

```
}
```

14. Troubleshooting

"No signal" immediately after connecting

- Ensure the headset is on your forehead, not just held in your hand
- The metal electrode must touch your skin — push hair aside
- Moisten the ear clip contact slightly
- Try `.poorSignal` simulator mode to confirm your UI handles `.noSignal` correctly

BLE connection times out on macOS

- The device may already be connected to another host — power-cycle the headset
- Try moving within 1 meter of the headset
- Disable and re-enable Bluetooth in macOS System Settings
- If the issue persists, try BT Classic on macOS: `sdk.connect("MindWave Mobile", mode: .btClassic)`

BT Classic "device not found" on macOS

- Open System Settings → Bluetooth and pair the MindWave Mobile before connecting
- The device must appear as Paired (not just Discovered) in the macOS Bluetooth list
- Ensure `com.apple.security.device.bluetooth` is in your app's entitlements

@MainActor concurrency warnings

NeuroSkySdk is isolated to @MainActor. If you see concurrency warnings, ensure you call it from a @MainActor context:

```
// Correct
Task { @MainActor in
    try await sdk.connect("MindWave Mobile")
}

// Also correct in SwiftUI
.task {
    try await sdk.connect("MindWave Mobile")
}
```

iOS Simulator

CoreBluetooth does not work in the iOS Simulator. Use `NeuroSkySdk(simulator:)` for development without a physical device:

```
#if targetEnvironment(simulator)
    let sdk = NeuroSkySdk(simulator: .focused)
#else
    let sdk = NeuroSkySdk()
#endif
```

15. Testing

The SDK ships with two test suites:

Unit tests — ThinkGearParserTests

Tests for the packet parser covering:

- 0xEA packet: attention, meditation, poorSignal extraction
- 0xEB packet: EEG frequency band 1/2
- Raw EEG: 10-sample parsing with sign correction
- Handshake packet: 20-byte structure and checksum

Integration tests — SimulatorIntegrationTests

Virtual device tests covering the full lifecycle:

Test	Verifies
test_connect_yieldsConnectingThenConnected	State sequence on connect
test_disconnect_yieldsDisconnected	State on disconnect
test_focused_attentionInExpectedRange	.focused mode value ranges
test_relaxed_meditationInExpectedRange	.relaxed mode value ranges
test_poorSignal_signalQualityIsNoSignal	.poorSignal mode + .noSignal quality
test_rawEeg_has10SamplesPerPacket	10 samples per packet
test_rawEeg_samplesInvalidRange	Sample range (-512~512)
test_receivesMultiplePacketsOverTime	≥ 2 packets in 4 seconds
test_sendCommand_doesNotThrow	All commands succeed

Run tests

```
swift test
```

Or, from within Xcode: Product → Test (⌘U).

16. API Reference

NeuroSkySdk

The main entry point. @MainActor-isolated for safe UI updates.

```
@MainActor
public final class NeuroSkySdk {

    /// Stream of EEG data packets from the headset
    public let dataStream: AsyncStream<BrainWaveData>

    /// Stream of connection state changes
    public let stateStream: AsyncStream<ConnectionState>

    /// Initialize for real device connection
    public init()

    /// Initialize in simulator mode (no hardware required)
    public init(simulator mode: SimulatorTransport.Mode)

    /// Connect to headset by name or address.
    /// - iOS: BLE only.
    /// - mode: Transport to use. Defaults to .ble. Pass .btClassic on macOS for SPP.
    public func connect(_ deviceAddress: String,
                       mode: TransportMode = .ble) async throws

    /// Scan for a BLE peripheral by name and return its OS-assigned UUID string.
    /// Cache the result to skip scanning on subsequent launches.
    public func findDeviceIdentifier(_ deviceName: String,
                                    timeout: TimeInterval = 10) async -> String?

    /// Disconnect from headset
    public func disconnect() async

    /// Send a raw command byte to the headset
    public func sendCommand(_ command: UInt8) async throws

    // Convenience methods
    public func startRawEeg() async throws
    public func stopRawEeg() async throws
    public func setNotch50Hz() async throws
    public func setNotch60Hz() async throws
}
```


ConnectionState

```
public enum ConnectionState: Equatable {
    case scanning
    case connecting
    case connected
    case disconnected
    case error(Error)
}
```

BrainWaveData

```
public struct BrainWaveData {
    public let poorSignal: Int
    public let attention: Int
    public let meditation: Int
    public let delta: Int
    public let theta: Int
    public let lowAlpha: Int
    public let highAlpha: Int
    public let lowBeta: Int
    public let highBeta: Int
    public let lowGamma: Int
    public let midGamma: Int
    public let rawEeg: [Int]
    public var signalQuality: SignalQuality
}
```

SignalQuality

```
public enum SignalQuality: Equatable {
    case good // poorSignal == 0
    case fair // poorSignal 1~50
    case poor // poorSignal 51~199
    case noSignal // poorSignal == 200
}
```

SimulatorTransport.Mode

```
public enum Mode {
    case random // randomized values
    case focused // high attention, mid meditation
    case relaxed // low attention, high meditation
    case poorSignal // poorSignal = 200, all zeros
}
```

NeuroSkyCommand

```
public enum NeuroSkyCommand {
    public static let startRawEeg: UInt8    // 0x15
    public static let stopRawEeg: UInt8     // 0x16
    public static let startEsense: UInt8    // 0x17
    public static let stopEsense: UInt8     // 0x18
    public static let notch50Hz: UInt8     // 0x1B
    public static let notch60Hz: UInt8     // 0x1C
}
```

Error types

```
public enum BLEError: Error {
    case bluetoothUnavailable    // Bluetooth is off or restricted
    case connectionFailed       // GATT connection failed
    case deviceNotFound          // No matching peripheral found during scan
}

public enum BLError: Error {
    case deviceNotFound          // Not found in paired device list (macOS)
    case connectionFailed       // RFCOMM channel failed to open
}

public enum TransportError: Error {
    case btClassicNotAvailableOniOS // .btClassic mode is macOS only
}
```

TransportMode

```
public enum TransportMode {
    case ble          // CoreBluetooth GATT – iOS + macOS, no pairing required (default)
    case btClassic    // IOBluetooth RFCOMM SPP – macOS only, requires pairing
}
```

NeuroSky MindWave Mobile Apple SDK v1.0.0 · Apache License 2.0 · github.com/nsk-bci/mindwave-sdk-apple